Comparison of automated methods for AI Video Detection

Group Report for COMP5530

AYOMIDE SORUNA*, ALEX NORTHAM*, CHIEN-WEI TUNG*, HSIN JEE SUM*, and ABBIE HEWITT*, University of Leeds, United Kingdom

MUHAMMAD ZEESHAN BABAR†, University of Leeds, United Kingdom

Reliably distinguishing real video from AI-generated content has become increasingly difficult as models such as Sora have made high-fidelity video synthesis widely accessible. This paper presents a comparative study of eight architectures for AI-generated video detection on the IVY-Fake benchmark, spanning spatial aggregation, recurrent temporal modelling, factorised spatiotemporal convolution, transformer-based attention, and geometric trajectory analysis, alongside a Random Forest ensemble that combines their predictions. All models are evaluated under a strict open-set protocol in which test sources are entirely unseen during training.

The LRCN achieves the strongest individual test accuracy (77.80%, AUC 0.9186), while the Random Forest ensemble achieves the highest macro-F1 (0.7252) and recall (0.6214) by exploiting complementary errors across architectures. All models exhibit a consistent val/test generalisation gap, which we attribute to source-signature learning rather than classical overfitting: models learn to recognise specific training generators rather than a general concept of AI-generated content. These results establish a baseline on IVY-Fake and identify training-source diversity, not architectural choice, as the primary bottleneck for cross-source generalisation.

1 INTRODUCTION

The rapid democratisation of high quality artificially generated videos has fundamentally altered the landscape of digital videos. This has brought with it increasing concerns regarding digital authenticity, information integrity, and security. As the ability of modern models to produce convincing content increases, the visual boundary between what is real and what is fake continues to blur. Consequently the development of detection measures that are robust and scalable has transitioned from an academic curiosity to a societal necessity.

Early detection methods primarily focused on identifying spatial inconsistencies within individual frames, such as asymmetrical features, unnatural lighting or blending errors. Modern video generation models have largely overcome these static imperfections, and so the new frontier of deepfake detection relies heavily on identifying the unnatural ways in which generated physics, lighting and geometry evolve over time, i.e. extending detection into the temporal domain.

To address this problem, this project presents a comparative study evaluating the performance of a number of different model types trained for AI video detection. Using the extensive IVY-Fake dataset, we designed and implemented a classification framework for benchmarking these different architectures.

Our evaluation ranges from computationally efficient spatial aggregators (Frame Mean Pooling) and recurrent architectures (LRCN), to explicitly spatiotemporal networks (custom 3D and factorised 2D+1D CNNs), and transformer and geometric trajectory models (Video Swin Transformer and ReStraV). An ensmeble model that combines all those aforementioned with a Random Forest Classifier used in ensemble model is also assessed.

A key design principle of our evaluation is the use of an open-set test protocol: all test sources are entirely absent from training, directly measuring cross-source generalisation rather than in-distribution performance. This is the most practically relevant evaluation setting, as real-world deployment requires detecting fakes from generators that did not exist at training time.

Our results show that all evaluated architectures achieve high validation accuracy but exhibit a consistent drop at test time, with test accuracy ranging from 70% to 78% across models. The LRCN performs strongest overall, achieving 77.80% test accuracy and an AUC of 0.9186.

Per-source analysis reveals that this drop stems from source-signature learning: models learn to recognise specific generators seen during training rather than a general concept of AI-generated content. This failure mode is consistent across all architectures regardless of design complexity or the use of pretrained weights, and represents the central finding of this work.

2 BACKGROUND RESEARCH

Research on detecting fake images/videos has been around as long as the ability to create them. In the mid 2010s, deepfake technology became widely available. This involves replacing the face of an actor with the face of a second actor [Marissa Koopman 2018]. As a result a great deal of research was done on developing methods of detecting these deepfakes from real pictures. These were largely spatial in nature, identifying inconsistencies within individual frames such as asymetrical facial features or unnatural lighting.

Modern generative models however have largely overcome these static imperfections, meaning it is increasingly difficult to distinguish real from synthetic content on the basis of spatial analysis alone. The new frontier of AI video detection therefore relies on identifying the unnatural ways in which generated content evolves over time, extending detection into the temporal domain.

*Sharing first authorship

†Also with the role of Supervisor.

Authors' addresses: Ayomide Soruna, email@leeds.ac.uk; Alex Northam, sc222an@leeds.ac.uk; Chien-Wei Tung, fy19cwt@leeds.ac.uk; Hsin Jee Sum, sc22hjs@leeds.ac.uk; Abbie Hewitt, sc22ah3@leeds.ac.uk, University of Leeds, Woodhouse, Leeds, West Yorkshire, United Kingdom, LS2 9JT; Muhammad Zeeshan Babar, M.Z.Babar@leeds.ac.uk, University of Leeds, Woodhouse, Leeds, West Yorkshire, United Kingdom, LS2 9JT.

This section reviews the major architectural approaches to this problem, tracing the progression from purely spatial models through increasingly sophisticated temporal representations.

2.1 Feature Extraction and Spatial Representations

Raw video pixels are far too noisy for a classifier to be able to understand, so a prerequisite for any video classification system is the ability to extract meaningful representations from raw visual input.

Convolutional Neural Networks (CNNs) have been the dominant approach to this since [Lecun et al. 1998] demonstrated that locally connected architectures could exploit the spatial structure of images in ways that fully connected networks could not. The ResNet architecture, a type of CNN, is of particular importance to this project. ResNet acts as a hierarchy of filters, with the first layer finding generic features that could apply to a variety of datasets [Jason Yosinski 2014].

This transferability is directly exploited in several models evaluated in this project, where pretrained ResNet-18 backbones are used as frame-level feature extractors, avoiding the need to learn low-level visual features from scratch on the IVY-Fake dataset alone.

Early approaches to synthetic video detection relied heavily on frame by frame analysis, using standard 2D CNNs to identify spatial artefacts such as unnatural edge formations and colour inconsistencies [Arpan Mahara 2025]. These solutions were computationally efficient but blind to the flow of time, treating each frame as a standalone image. As generative models have improved so has the spatial fidelity of the images and videos they can create, leading to the detection of temporal inconsistencies becoming increasingly more important in accurately classifying videos.

A consistent challenge across all detection approach is, cross-dataset generalisation: models trained on one generation method tend to overfit to its specific artefacts and fail on unseen generators [Rössler et al. 2019; Wang et al. 2019]. This limitation is directly relevant to the IVY-Fake benchmark, where test sources are entirely disjoint from training, and motivates the per-source analysis reported in Section 5.

2.2 Temporal Modelling

2.2.1 Recurrent Approaches. One of the earliest and most natural ways to incorporate temporal information into a CNN-based video classifier is through recurrent architectures. [Donahue et al. 2015] introduced the Long-term Recurrent Convolutional Network (LRCN), which combines a 2D CNN backbone with a Long Short-Term Memory (LSTM) network. The visual input is processed by the CNN to produce a fixed length feature vector and the sequence of these is passed to the LSTM, which models temporal dependencies across the sequence. [Hochreiter and Schmidhuber 1997] demonstrated that LSTMs address the vanishing gradient problem, a fundamental limitation of standard recurrent networks, by maintaining a memory cell whose contents are regulated by learned gates, allowing the network to retain relevant information over sequences in excess of 1000 time steps.

In the context of synthetic video detection, this is significant as temporal inconsistencies may manifest across many frames rather than between adjacent ones, requiring this kind of long range memory. The key limitation of this approach, however, is that spatial and temporal processing remain decoupled: the CNN extracts features from each frame independently, and the LSTM then reasons about the sequence of those features rather than learning joint spatiotemporal representations.

2.2.2 Convolutional Networks. To address this decoupling, [Tran et al. 2014] proposed learning spatiotemporal features jointly using 3D convolutional networks, arguing that "3D ConvNets are well-suited for spatiotemporal feature learning" and that compared to 2D CNNs, the ability to perform 3D convolutions and 3D pooling operations allows the effective modelling of temporal information.

Rather than applying convolutions only across the spatial dimensions of each frame, a 3D convolution slides a kernel across height, width and time simultaneously, meaning that a single neuron in a deeper layer is influenced by a spatiotemporal patch of the original video rather than a purely spatial one. This allows the network to detect motion patterns and temporal inconsistencies that only manifest across multiple frames, properties that are particularly relevant for detecting generative artefacts such as unnatural object motion or temporal flickering.

A practical limitation of 3D convolutions is their parameter cost. A 3D kernel of size t×d×d contains roughly 50% more parameters than an equivalent 2D kernel, which motivated subsequent work on more efficient spatiotemporal representations.

2.2.3 Factorised Spatiotemporal Convolutions. [Tran et al. 2018] proposed a factorised alternative to full 3D convolution. The (2+1)D convolutional block decomposes each 3D convolution into a 2D spatial convolution followed by a 1D temporal one.

First, an additional ReLU non-linearity is introduced between the spatial and temporal passes, which they show "effectively doubles the number of nonlinearities compared to a network using full 3D convolutions for the same number of parameters", increasing representational capacity without increasing cost.

Second, decoupling spatial and temporal optimisation produces lower training and test loss than an equivalent full 3D network, suggesting the factorisation makes the learning problem easier to solve.

2.2.4 Attention Based Approaches. The introduction of the Transformer architecture and its application to vision tasks opened a new direction for video understanding. Unlike convolutional approaches which capture features through learned filters with a fixed receptive field, transformers compute attention scores between all tokens within a defined window, allowing the model to learn flexible relationships between any regions it finds relevant rather than being constrained to a fixed neighbourhood.

[Liu et al. 2021] extended this to the video domain with the Video Swin Transformer, proposing to compute self-attention within local 3D windows of spatiotemporal patches rather than globally across the entire video. The authors argue that "a global self-attention

module would thus be unsuitable for video tasks as this would lead to enormous computation and memory costs", and instead "advocate an inductive bias of locality in video Transformers, which leads to a better speed-accuracy trade-off", noting that "full spatiotemporal self-attention can be well-approximated by self-attention computed locally, at a significant saving in computation."

The shifted window mechanism allows information to flow between adjacent windows across layers by offsetting the window partition in alternating transformer blocks, giving the model an effectively large receptive field while keeping computational cost linear in the number of tokens.

For synthetic video detection specifically, the attention mechanism offers a potentially meaningful advantage over convolutional approaches. Rather than applying the same fixed filters everywhere, the model can learn to attend dynamically to the specific spatial and temporal regions where generative artefacts tend to appear, such as facial boundaries or regions of temporal inconsistency.

2.2.5 Trajectory Based Approaches. All of the architectures described above classify videos on the basis of their visual appearance, either frame by frame or across spatiotemporal volumes. A fundamentally different approach is proposed by [Internò et al. 2026] with ReStraV (Representation Straightening for Video), which instead classifies the geometric shape of a video's trajectory through neural representation space. The method is motivated by the perceptual straightening hypothesis, the observation that "natural inputs map to straight paths in neural representations while unnatural sequences form curved trajectories".

[Internò et al. 2026] leverage this property for synthetic video detection, arguing that because DINOv2, which is a self-supervised vision transformer trained on large quantities of natural image data, has learned a representation space that reflects the statistics of the natural world, real video trajectories through this space should be geometrically smooth, while AI generated videos, which do not follow the same physical and perceptual laws, should deviate from this pattern in measurable ways.

Frames are encoded by DINOv2 to create a series of vectors, from which 21 geometric features are derived: seven stepwise euclidean distances between frames, six angles between steps, and eight summary statistics. These features are then passed to a lightweight 2 layer MLP classifier.

2.2.6 Ensemble Methods.
Ensemble methods improve predictive performance by combining models whose errors are partially independent since when one model fails on a given example others may succeed and aggregation smooths out individual weaknesses. Ensemble methods have been shown to take advantage of multiple learned models to achieve a much improved predictive performance compared to each model individually [Garouani et al. 2025]. A stacked ensemble learning model can combine similar models by training a meta-learner on the predictions of each base model. They combine the predictions of the base models to improve generalisation and robustness of the final model. The randomness in the forests yields decision trees with decoupled prediction errors that can later be cancelled out, achieving reduced variance by combining diverse trees. [Pedregosa et al. 2011]

Ensembles reduce variance and bias since they consistently outperform standalone models when their group of models is diverse [Breiman 1996]. In deepfake detection, diversity arises naturally since different architectures capture different artefacts. Frame level models are sensitive to spatial inconsistencies, recurrent models are sensitive to temporal inconsistencies, and volumetric models are sensitive to joint spatio-temporal inconsistencies. Due to each model specialising to identify different artefacts, the trained models are diverse, so an ensemble method is desired to mitigate the limitations of any individual model.

2.3 Summary

The architectures reviewed in this section represent three broad paradigms for video understanding, each making different assumptions about where the signal for detecting AI-generated content lies.

Frame-based models treat videos as collections of independent images and aggregate spatial features over time, making them efficient but insensitive to temporal dynamics. Clip-based models, including 3D CNNs, (2+1)D CNNs, and the Video Swin Transformer, process spatiotemporal volumes jointly, capturing motion and temporal consistency at the cost of greater computational complexity. Trajectory-based models such as ReStraV bypass visual content entirely, instead reasoning about the geometry of feature trajectories.

This project evaluates representatives of all three paradigms on the IVY-Fake dataset, with the goal of determining which approach best captures the temporal and spatial properties that distinguish real from AI generated video.

3 METHOD

3.1 Dataset and Data Organisation

This study was conducted using the IVY-Fake dataset [Changjing Jiang 2025], a large-scale collection of real and AI-generated videos. The dataset includes content generated from multiple video synthesis systems alongside real-world video sources, with each video labelled as either real or fake, enabling supervised binary classification.

The dataset is organised hierarchically by split, source, and label. Videos are grouped into training, validation, and test sets, with each split containing subdirectories corresponding to different generative models or real-world collections. The training split contains six generative sources (DynamicCrafter, Latte, OpenSora, SEINE, ZeroScope, and Pika) alongside two real-world collections (Kinetics-400 and Yonku_1M_10s).

Crucially, the test split introduces entirely unseen sources: twelve generative models and two real-world collections not present during training, directly measuring cross-source generalisation rather than in-distribution performance. The dataset splits are pre-defined and curated to ensure data quality and experimental integrity; invalid or corrupted samples are removed from all, and the splits are designed to prevent data leakage between training and evaluation sets.

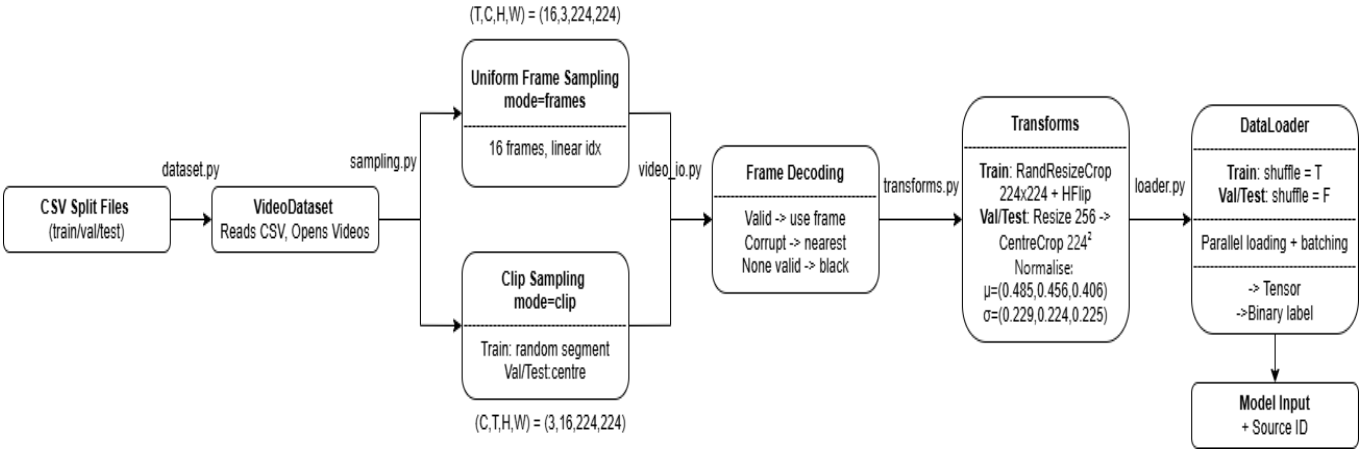


Fig. 1. Video preprocessing pipeline.

3.2 Data Processing Pipeline

The data processing pipeline converts raw video files into structured tensor representations suitable for deep learning models. This process consists of video decoding, temporal sampling, and frame-level transformations.

3.2.1 Video Decoding.

Video data is decoded using OpenCV. Each video file is accessed via a `VideoCapture` object, allowing individual frames to be retrieved based on their index without requiring full video decoding into memory, which is essential given the scale of the dataset. Corrupt or unreadable frames are handled gracefully by substituting the nearest successfully decoded frame, or a black frame if no valid frame has been decoded yet.

3.2.2 Sampling Strategies.

Two sampling strategies are implemented to accommodate different modelling approaches. In frame mode, 16 frames are sampled uniformly across the full video duration using linearly spaced indices, providing global temporal coverage regardless of video length. In clip mode, a contiguous sequence of 16 frames is extracted from a randomly selected temporal segment during training, or from the video centre during validation and testing, preserving short-term temporal continuity [Tran et al. 2015].

These two strategies reflect different assumptions about the importance of temporal order and are used to support different model architectures.

3.2.3 Data Transformations.

Each sampled frame undergoes a series of transformations prior to being passed to the model. These transformations are implemented using the PyTorch `torchvision.transforms` library.

For training data, frames are randomly resized and cropped to 224×224 pixels and randomly flipped horizontally, introducing variability to improve robustness and reduce overfitting [Shorten and Khoshgoftaar 2019]. For validation and test data, deterministic transformations are applied, frames are resized to 256 pixels on the shorter side then centre-cropped to 224×224. All frames are normalised using ImageNet statistics (mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)), ensuring compatibility with pretrained backbones.

3.2.4 Data Representation.

Following transformation, frames are stacked into tensor representations that depend on the sampling mode and model requirements. For frame-based models, the data is represented as a sequence of individual frames with shape:

((T,C,H,W))

where **T** is the number of frames, **C** is the number of channels (RGB), and **H,W** denote spatial dimensions. For clip-based models, the data is arranged as:

(C,T,H,W)

This format aligns with the expected input structure of 3D convolutional networks, which operate jointly across spatial and temporal dimensions.

3.2.5 Data Loading.

The processed data is encapsulated within a custom PyTorch `VideoDataset` class [et al. 2019], which handles video decoding, frame sampling, and transformation. This is then wrapped by a PyTorch `DataLoader`, which provides batching, shuffling, and parallel data loading functionality. The DataLoader is configured with `shuffle=True` for training and `shuffle=False` for validation and test, and returns input tensors, binary labels, and per-video metadata including the source identifier, enabling per-source accuracy analysis during evaluation.

3.3 Aire HPC

All models detailed below were trained on the infrastructure shown in Table 1. Access to this infrastructure was provided through a login by the University of Leeds. The default training hyperparameters can be viewed in Table 2, and specific hyperparameters that were tuned can be found in each model's individual methodology.

Table 1. GPU Node Specifications

Component	Specification
Server Model	Dell PowerEdge R7615
Node Count	28
GPUs per Node	3 × NVIDIA L40S 48GB (PCIe)
Total GPU Cards	84
CPU	AMD EPYC 9254 (Genoa-X), 24 cores, 2.9GHz
System Memory	256GB DDR5-4800
Local Storage	2 × 480GB M.2 NVMe

Table 2. Default Training Hyperparameters and Configuration Arguments

Argument	Default Value	Description
Data Configuration		
`-mode`	*required*	Input mode (`frames` or `clip`)
`-dataset_root`	*<dataset_root>*	Root directory of the dataset
`-train_csv`	*<train_csv>*	Path to training split CSV
`-val_csv`	*<val_csv>*	Path to validation split CSV
`-test_csv`	*<test_csv>*	Path to test split CSV
`-image_size`	`224`	Input image resolution (px)
Temporal Configuration		
`-n_frames`	`16`	Number of frames sampled per video
`-clip_len`	`16`	Number of frames per clip
Model & Training Configuration		
`-backbone`	*<backbone>*	CNN backbone architecture
`-batch_size`	`8`	Training batch size
`-num_workers`	`4`	DataLoader worker threads
`-epochs`	`10`	Number of training epochs
`-lr`	`3e-4`	Learning rate
`-weight_decay`	`1e-4`	L2 regularisation weight decay
Output Configuration		
`-out`	*<checkpoint_path>*	Checkpoint path (best val loss)
`-acc_out`	*<checkpoint_path>*	Checkpoint path (best accuracy)

3.4 Clip Based Models

3.4.1 Clip Baseline.

The clip-based baseline uses a pretrained 3D ResNet-18 (R3D-18) [Tran et al. 2014], pretrained on the Kinetics action recognition dataset. Unlike frame-based models, this architecture processes spatial and temporal information jointly via 3D convolutions.The final fully connected layer is replaced with a linear classifier producing a single logit for binary classification. Training uses AdamW with a learning rate of 3×10^{-4} and weight decay of 1×10^{-4}.

Early layers of the backbone (stem, layer1, layer2) are frozen to preserve pretrained spatiotemporal features, with only deeper layers fine-tuned. Weighted binary cross-entropy loss is applied with pos_weight computed from the training CSV. Mixed precision training is used on CUDA via `torch.amp.autocast`. A latest checkpoint is saved every epoch to support job resumption on the cluster, and a separate best checkpoint is saved whenever validation loss improves.

3.4.2 3DCNN.

The custom 3D CNN is trained from scratch rather than fine-tuned from pretrained weights, making it the only clip-based model in this study without transfer learning. The architecture consists of four sequential `Conv3DBlock` modules, each containing a 3D convolution, batch normalisation, ReLU activation, and max pooling, progressively expanding channel depth from 3 to 32, 64, 128, and 256. The first block uses a spatial-only pooling kernel of (1,2,2) to preserve temporal resolution in early layers, with subsequent blocks applying full (2,2,2) spatiotemporal pooling. The resulting feature volume is collapsed to a single vector using global average pooling, and this is then passed to a two-layer MLP classifier with a hidden size of 128 and dropout of 0.5.

Training uses Adam with a learning rate of 3×10^{-4} and weight decay of 1×10^{-4}, without weighted loss or mixed precision. A `ReduceLROnPlateau` scheduler halves the learning rate if validation loss does not improve for 3 consecutive epochs, and early stopping terminates training after 7 epochs without improvement. Two checkpoints are maintained: one for best validation loss and one for best validation accuracy.

3.4.3 2D + 1D CNN.

Two (2+1)D CNN variants were implemented, both factorising each 3D convolution into a 2D spatial convolution over the spatial dimensions followed by a 1D temporal convolution over the time axis [Tran et al. 2018]. The intermediate channel size between the two convolutions is computed as

$$M_i = \left\lfloor \frac{k_t \cdot k_s^2 \cdot c_{in} \cdot c_{out}}{k_s^2 \cdot c_{in} + k_t \cdot c_{out}} \right\rfloor \tag{1}$$

where k_t and k_s are the temporal and spatial kernel sizes respectively. This is computed to match the parameter count of the equivalent full 3D convolution [Tran et al. 2017].

The simpler variant stacks four `TwoPlusOneDBlock` modules, each consisting of a spatial convolution, batch normalisation, ReLU activation, temporal convolution, batch normalisation, ReLU activation, and a max-pool layer with kernel size (1,2,2), (2,2,2), (2,2,2), and (2,2,2) respectively. The channel depth expands from $3 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$. A global average pool collapses the spatio-temporal dimensions, followed by a two-layer MLP classifier ($256 \rightarrow 128 \rightarrow 1$) with ReLU activation and a dropout of 0.5.

The Residual (2+1)D CNN (R(2+1)D CNN) has skip connections via `Residual2Plus1DBlock`, each of which contains two factorised (2+1)D convolutions with a residual addition. When the channel count changes, the residual network uses a $1 \times 1 \times 1$ convolution with batch normalisation. It opens with a (2+1)D stem; a spatial $(1, 7, 7)$ convolution to 45 channels, a temporal $(3, 1, 1)$ convolution to 64 channels, and a max-pool with kernel size $(1, 3, 3)$ and a stride of $(1, 2, 2)$. A stride in a convolutional neural network is the number of pixels that a convolutional filter shifts across an image during convolution. The stem is before four residual stages with $[2, 2, 2, 2]$ blocks expanding the channel depth from $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$. The third and fourth stage apply a temporal stride of 2 in addition to the spatial stride of 2. A global average pool and linear classifier with a dropout of 0.5 produce the final prediction logit. Weights are initialised using Kaiming normal initialisation for convolutional layers, ones and zeros for batch normalisation, and normal distribution ($\sigma = 0.01$) for linear layers.

Both (2+1)D CNN variants are trained with the Adam optimiser at a learning rate of 3×10^{-4} and weight decay of 1×10^{-4}, using binary cross-entropy with logits loss. A `ReduceLROnPlateau` scheduler is implemented to halve the learning rate after 3 epochs if there is no improvement in validation loss, and training is stopped early after 7 epochs without efficacious performance. Two checkpoints are maintained per run: one for best validation loss and one for best validation accuracy.

3.4.4 Video Swin Transformer.

The Video Swin Transformer [Liu et al. 2022] uses PyTorch's pretrained `swin3d_b` (Base) variant from `torchvision.models.video`, originally trained on Kinetics-400 for action recognition. The model applies 3D shifted-window self-attention as described in Section 2, processing clips of shape $(3, 16, 224, 224)$ through four hierarchical stages with a $[2, 2, 18, 2]$ block configuration and approximately 88M trainable parameters. The Base variant is selected over the Tiny and Small alternatives to maximise spatiotemporal feature capacity within the available A100 GPU memory budget under mixed precision. The pretrained classification head is replaced with a single linear layer producing one logit for binary real/fake classification.

Training uses `BCEWithLogitsLoss` with `pos_weight` = 1.23, computed as the negative-to-positive ratio from the training split (49,500 real, 40,275 fake). The AdamW optimiser is applied with decoupled learning rates of 5×10^{-5} for the pretrained backbone and 1×10^{-3} for the classifier head, with weight decay of 1×10^{-4}. The backbone is frozen for the first epoch to allow the classifier head to converge before joint fine-tuning. A `ReduceLROnPlateau` scheduler halves both learning rates when validation loss stalls, and early stopping is applied with a patience of 7 epochs; in practice, training terminated at epoch 13 of a scheduled 20. Mixed precision training via `torch.amp.autocast` is used on CUDA to fit the model into per-GPU memory at batch size 8.

The decision threshold is tuned on the validation set to maximise macro-F1, with the selected threshold used at test time. The resulting predictions feed into the Random Forest ensemble described in Section 3.8.

3.5 Frame Based Models

3.5.1 Frame Baseline.

The frame-based baseline applies a pretrained ResNet-18 to each of the 16 uniformly sampled frames independently, mean-pools the resulting feature vectors across the time dimension, and passes the pooled representation through a linear classifier. In this approach, sometimes referred to as late fusion, the order of frames is entirely discarded. It serves as a lower bound for temporal modelling: any accuracy gap between this model and the LRCN can be attributed specifically to the LSTM's ability to capture sequential dependencies. Training follows the same configuration as the clip baseline: AdamW, lr=3×10^{-4}, early layer freezing, weighted loss, and mixed precision.

3.5.2 LRCN.

The Frame-LSTM model, implemented as an LRCN, combines a pretrained ResNet-18 backbone with a single-layer unidirectional LSTM for temporal aggregation. Each of the 16 uniformly sampled frames is independently encoded by the ResNet-18 backbone, with the final classification layer replaced by an identity mapping, producing a 512-dimensional feature vector. The sequence of 16 feature vectors is passed to the LSTM with hidden size 512, and the final hidden state is fed to a linear classifier producing a single logit. Early layers of the ResNet-18 backbone (stem, layer1, layer2) are frozen during training to preserve pretrained spatial features, with only deeper layers and the LSTM updated.

Training uses AdamW with a learning rate of 3×10^{-4} and weight decay of 1×10^{-4}. Weighted binary cross-entropy loss is applied with pos_weight computed from the training CSV. Mixed precision training is used on CUDA. A dual checkpoint system saves a latest checkpoint every epoch to support resumption from cluster timeouts, and a best checkpoint whenever validation loss improves.

3.6 Trajectory Based Models

3.6.1 ReStraV.

The ReStraV model is a lightweight embedding-trajectory-based model for AI-generated video detection. Following the trajectory-based approach described in Section 2, sampled frames are encoded using a frozen pretrained DINO-based vision backbone to obtain frame-level embeddings. In the baseline implementation, DINOv2 is used as the feature extractor, and the resulting embedding sequence is converted into a 21-dimensional geometric trajectory feature vector consisting of stepwise distances, turning angles, and summary statistics. These features are passed to a lightweight two-layer MLP classifier with hidden dimensions 128 and 64, dropout of 0.3, and a single-logit output.

To improve the baseline pipeline, the final ReStraV configuration replaces DINOv2 with DINOv3, reduces the sampled frame count from 24 to 16, applies adaptive multi-clip inference for longer videos, uses max aggregation for clip-level predictions, applies class weighting, and selects the final decision threshold based on validation performance rather than the default value of 0.5.

3.7 Training and Testing Pipeline

All models share a common training structure. Mini-batch stochastic gradient descent is performed using an optimiser from the Adam family: standard Adam for the 3D CNN and (2+1)D CNN, and AdamW for the clip baseline, frame baseline, LRCN, and Video Swin Transformer. Binary cross-entropy with logits (`nn.BCEWithLogitsLoss`) is used as the loss function across all models. All models use an epoch-based training loop and save a checkpoint when validation loss improves. Where implemented, GPU acceleration is used via CUDA with non-blocking data transfers. Models differ in their use of weighted loss, mixed precision training, learning rate scheduling, and early stopping, as described in the individual model sections above. All training was conducted on the Aire HPC cluster using a single GPU per job via SLURM.

Algorithm 1 Model Training Loop

Require: Training set $\mathcal{D}_{train}$, validation set $\mathcal{D}_{val}$, model M, optimiser θ, loss function $\mathcal{L}$
Ensure: Best model checkpoint M^*

$\mathcal{L}^* \leftarrow \infty$
for $epoch = 1$ **to** E **do**
  M.train()
  for each batch $(x, y) \in \mathcal{D}_{train}$ **do**
    $\hat{y} \leftarrow M(x)$ ▷ raw logit
    $\ell \leftarrow \mathcal{L}(\hat{y},\ y)$
    Backpropagate ℓ; update θ
  end for
  M.eval()
  $\mathcal{L}_{val} \leftarrow \mathcal{L}(M(\mathcal{D}_{val}))$
  if $\mathcal{L}_{val} < \mathcal{L}^*$ **then**
    $\mathcal{L}^* \leftarrow \mathcal{L}_{val}$; save M^*
  end if
end for
return M^*

3.8 Model of Models

The ensemble combines the predictions of all eight base models through a Random Forest meta-classifier, motivated by the hypothesis that architectures with different inductive biases (spatial, recurrent, spatio-temporal, attention-based, and trajectory-based) make systematically different errors and can therefore be combined to produce more robust predictions than any single model. This hypothesis is examined empirically in Section 4.

Because the ensemble operates on the trained outputs of all eight base models, every methodological choice described in Sections 3.4 – 3.7 propagates into the meta-classifier's input space. We therefore evaluate the ensemble both as a comparison against the strongest individual architectures and as a probe of whether the paradigm diversity built into the eight-model design produces the complementarity that the hypothesis above predicts.

The ensemble model was implemented in Python using `PyTorch` for probability extraction and `scikit-learn` for the meta-learner as part of the random forest classifier. All models were loaded from their best validation-loss checkpoints and set to evaluation mode. To extract the probability scores, the logits of each model were passed through a sigmoid function to produce a scalar probability of $[0, 1]$ per model. The eight probabilities form the basis of the meta-classifier's input.

3.8.1 *Feature Construction.*

For each video, the eight base-model probabilities $\{\hat{y}_1, \ldots, \hat{y}_8\}$ are augmented with three aggregate statistics computed across the models: the mean $\bar{y}$, the maximum $\max(\hat{y})$, and the standard deviation $\sigma(\hat{y})$. This produces an 11-dimensional feature vector per video. The standard deviation serves as an explicit measure of inter-model disagreement, motivated by the assumption that videos lying clearly on one side of the real/fake boundary will elicit consistent predictions across architectures, while ambiguous or out-of-distribution inputs will produce greater variance. While a Random Forest can in principle learn equivalent statistics through its splits, providing them as explicit features reduces the depth required to express such patterns and makes the resulting feature importances more directly interpretable. The full procedure is given in Algorithm 2.

3.8.2 *Random Forest Classifier.*

A random forest was chosen as the meta-learner due to its robustness to correlated input features and since it can provide feature importance scores. The random forest classifier was trained on the IVY-Fake validation set feature vectors. Using the validation set instead of the training set ensures that the base model's outputs are out-of-distribution compared to the models' own training data, since each individual model was trained on the training set. This helps to avoid data leakage and prevents over-fitting.

3.8.3 *Threshold Optimisation and Weighted Ensemble.*

As shown in Algorithm 2, a threshold of 0.42 was selected by sweeping thresholds from $0.1 - 0.9$ at intervals of 0.01 and selecting the best threshold that maximised the F1 score. To develop a weighted ensemble model, the random forest's feature importances were extracted and used as a weighted average ensemble. The weights were normalised to sum to one and were applied as a dot product against the base model's probability columns. The same threshold optimisation was applied independently to the weighted ensemble, also achieving an optimal threshold of 0.42. Feature importances were reported to identify which constituent models contributed the most to each final prediction.

Algorithm 2 Random Forest Ensemble Stacking

Require: Validation set $\mathcal{D}_{val}$, test set $\mathcal{D}_{test}$, pre-trained models $\mathcal{M} = \{M_1, \ldots, M_K\}$, checkpoints $\{\phi_1, \ldots, \phi_K\}$
Ensure: Ensemble predictions $\hat{y}_{test}$

for $k = 1$ **to** K **do**
  Load ϕ_k into M_k; move M_k to device
end for

for each $M_k \in \mathcal{M}$ **do**
  $p_k^{val} \leftarrow \sigma(M_k(\mathcal{D}_{val}))$ ▷ sigmoid probabilities
  $p_k^{test} \leftarrow \sigma(M_k(\mathcal{D}_{test}))$
end for

$X_{val} \leftarrow \left[p_1^{val}, \ldots, p_K^{val},\ \bar{p}^{val},\ \max(p^{val}),\ \mathrm{std}(p^{val})\right]$
$X_{test} \leftarrow \left[p_1^{test}, \ldots, p_K^{test},\ \bar{p}^{test},\ \max(p^{test}),\ \mathrm{std}(p^{test})\right]$

Train Random Forest $\mathcal{F}$ on $(X_{val},\ y_{val})$

$\hat{p}_{RF}^{val} \leftarrow \mathcal{F}$.predict_proba$(X_{test})$
$\tau^* \leftarrow_{\tau \in [0.1, 0.9]}\ F_1\left(y_{val},\ \hat{p}_{RF}^{val} \geq \tau\right)$

$\hat{p}_{RF}^{test} \leftarrow \mathcal{F}$.predict_proba$(X_{test})$
$\hat{y}_{test} \leftarrow \mathbf{1}\left[\hat{p}_{RF}^{test} \geq \tau^*\right]$

return $\hat{y}_{test}$

Model	Val Acc	Test Acc	F1	Precision	Recall	AUC
LRCN	0.9825	0.7780	0.7181	0.9774	0.5675	0.9186
ReStraV	0.8461	0.6999	0.6444	0.7867	0.5458	0.7531
VideoSwinTransformer	0.9926	0.7719	0.7098	0.9693	0.5599	0.8967
Residual (2+1)D CNN	0.9107	0.7366	0.6788	0.8651	0.5585	0.8109
3DCNN	0.8654	0.6433	0.5315	0.7690	0.4060	0.6872
(2+1)D CNN	0.9878	0.6905	0.5970	0.8499	0.4601	0.7428
Frame Baseline	0.9263	0.6813	0.6476	0.7209	0.5879	0.7258
Clip Baseline	0.9653	0.6458	0.5464	0.7548	0.4282	0.7056
Ensemble	—‡	0.7653	0.7252	0.9523	0.6214	—‡

Table 3. Validation and test performance across all models. Val Acc: validation accuracy. Test Acc: test accuracy. F1: macro-averaged F1 score. AUC: area under ROC curve. ‡The Ensemble operates over test-set model outputs and has no independent validation accuracy; AUC was not computed for the aggregated ensemble predictions.

Source	LRCN	ReStraV	VST	R(2+1)D CNN	3D CNN	(2+1)D CNN	Frame Baseline	Clip Baseline
Crafter	0.9158	0.9228	0.9789	0.9263	0.6877	0.9968	0.9158	0.8737
Gen2	0.9310	0.8897	0.8483	0.8897	0.7586	0.7345	0.9138	0.6759
HotShot	0.7600	0.0000	0.4600	0.2333	0.3200	0.2867	0.5733	0.3000
Kinetics-400-val	0.9872	0.7272	0.9944	0.9075	0.8754	0.9968	0.8650	0.9768
Lavie	0.6400	0.6400	0.6800	0.5100	0.3700	0.8800	0.6233	0.6433
MSR-VTT	0.9868	0.9788	0.9704	0.9196	0.8824	0.8419	0.6831	0.7471
ModelScope	0.8033	0.7300	0.9400	0.9200	0.7033	0.8833	0.8767	0.8067
MoonValley	0.9700	0.7467	0.9933	0.9367	0.8200	0.9633	0.9733	0.8400
MorphStudio	0.8562	0.7726	0.9565	0.9030	0.7224	0.9699	0.8930	0.8194
Show_1	0.6867	0.9800	0.3300	0.6300	0.4367	0.4633	0.7667	0.4567
Sora	0.7273	0.8182	0.6818	0.7273	0.3182	0.3182	0.5909	0.4545
WildScrape	0.6250	0.5777	0.6588	0.6385	0.5270	0.5777	0.6250	0.5000
Loki	0.5937	0.6322	0.5069	0.5909	0.4077	0.3237	0.5813	0.3264
self_craw	0.1543	0.2451	0.2380	0.2432	0.0830	0.0259	0.2276	0.0811

Table 4. Test performance across all sources

4 EVALUATION

4.1 Evaluation Strategy

All models are evaluated on the IVY-Fake benchmark , which is specifically designed to assess cross-source generalisation. The validation set shares sources with training, whilst the test set is a disjoint collection not used in any model development.

Train/Val sources comprise two real video collections (Kinetics-400 and Yonku_1M_10s) alongside six generative models (DynamicCrafter, Latte, OpenSora, SEINE, ZeroScope and Pika). The test set introduces two unseen real sources (Kinetics-400-val and MSR-VTT) and twelve unseen generative sources (Crafter, Gen2, HotShot, Lavie, ModelScope, MoonValley, MorphStudio, Show_1, Sora, WildScrape, Loki, and self_craw).

This open-set paradigm is critical, as it benchmarks a model's ability to detect synthetic content from generators entirely absent during training.

An important property of the dataset is how per-source metrics are interpreted. Every source is entirely single-class no source contains a mixture of real and fake videos. As a consequence, per-source AUC is undefined and is excluded from source level reporting. Per-source F1 for real sources is always zero regardless of model quality, since the absence of positive (fake) samples within a real source makes true positives impossible. Per-source accuracy is therefore the only meaningful metric for real sources, whilst both accuracy and F1 are informative for fake sources.

4.2 Metrics

Performance is measured using five complementary metrics. *Accuracy* measures the overall proportion of correctly classified videos. *Precision* and *recall* are reported for the fake (positive) class: precision captures how often a positive prediction is correct, whilst recall measures what fraction of true fakes are detected. *Macro-averaged F1* (MacroF1) is the harmonic mean of per class F1 scores, weighting real and fake classes equally irrespective of class imbalance. *Area under the ROC curve* (AUC) measures ranking ability across all possible thresholds, providing a threshold-independent view of discriminative performance. It represents the probability that a model

ranks a random positive example higher than a random negative one, with a score of 1.0 being perfect and 0.5 being random.

MacroF1 is the primary comparison metrics, for two reasons. First, accuracy is susceptible to inflation on majority class sources and does not require the model to generalise to rare or difficult generators. Second, the val/test accuracy gap tends to understate the true degradation in ranking ability. MacroF1 forces equal weight on both classes better identifying how well the models are at differentiating between real and fake videos.

4.3 Quantitative Results

The validation and test performance across all architectures is detailed in Table [3]. The complete data reveals a consistent and substantial degradation in generalisation across individual models, alongside compelling findings regarding ensemble methodologies.

Performance Degradation: All models achieve high validation accuracy, reflecting strong performance on seen source distributions. However at test time all models show a substantial drop with test accuracy ranging from 0.6433 (3DCNN) to 0.7780 (LRCN). This consistent val/test gap present across every architecture suggests that the degradation is not attributable to any single design decision, but is instead a consequence of evaluating on sources whose visual and temporal distributions differ from those seen during training.

Leading Standalone Architecture: The LRCN emerged as the strongest individual model, achieving the highest test accuracy (0.7780), F1 (0.7181), and AUC (0.9186) of all evaluated models.

Advantage of the Ensemble: The Random Forest Ensemble model introduces a critical shift in the performance landscape. While its raw test accuracy (0.7653) slightly trails the LRCN, the Ensemble achieves the highest MacroF1 score (0.7252) and the highest recall (0.6214) of any evaluated system.

Precision/Recall Imbalance: A pattern consistent across all models is the imbalance between precision and recall. Precision is high for every model, whilst recall is substantially lower (e.g., the 3DCNN achieved 0.7690 precision but 0.4060 recall). When a model predicts a video as fake it is usually correct, but a large proportion of fakes go undetected particularly from generators whose appearance differs from the training distribution.

4.4 Per-Source Analysis

Disaggregating the results by test source [4] exposes two distinct, systematic failure modes:

Blindness to Novel artefacts: Several fake sources are systematically misclassified as real by every model. HotShot is the most severe case: ReStraV achieves 0.00 accuracy, classifying every video in that source as real, whilst the Residual (2+1)D CNN achieves only 0.2333. The LRCN, despite being the strongest overall model, still only reaches 0.76 on HotShot, the best result of any model on that source. self_craw follows a similar pattern, with no model exceeding 0.25. Lavie and Loki are also consistently poorly detected across all architectures.

These failures share a common cause: the affected generators produce content whose visual and temporal properties are sufficiently unlike any training source that no model's learnt decision boundary extends to cover them, regardless of architecture.

Generalisation: Conversely, several unseen generators are detected with high accuracy. The LRCN leads on several sources, achieving 0.9872 on Kinetics-400-val, 0.9868 on MSR-VTT, 0.9700 on MoonValley, and 0.9310 on Gen2. Crafter, MoonValley, MorphStudio, and ModelScope are well-detected by the majority of models, plausibly because these generators share artefact characteristics with training sources such as DynamicCrafter, SEINE, and ZeroScope, enabling partial feature transfer despite the sources being technically unseen.

Real Video Sources: Both real test sources (Kinetics-400-val and MSR-VTT) achieve high accuracy across all models, with most results exceeding 0.90. As noted above, per-source F1 for these sources is zero by construction and carries no diagnostic meaning.

Taken together, the per-source results demonstrate that performance on a given test source is governed primarily by the visual and temporal similarity of that source to the training distribution, and not by whether a model uses temporal modelling, spatial aggregation, or pretrained representations. The LRCN's overall leadership does not translate into uniform superiority at the source level, reinforcing that no evaluated architecture has learnt a truly generalised concept of AI-generated video.

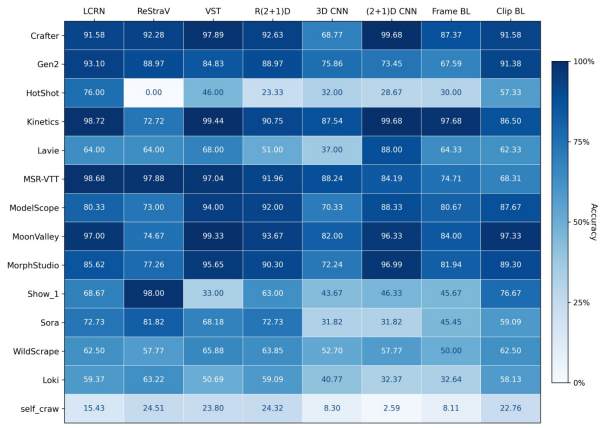


Fig. 2. Per-source test accuracy (%) across all models. The consistent light shading across self_craw, Loki, and WildScrape regardless of column confirms that failure on these sources is not model-specific but distribution-driven.

5 DISCUSSION

5.1 Source-Signature Learning as a Unifying Failure Mode

The most significant finding across this evaluation is not which architecture performs best, but that all architectures fail in the same way, as visualised in Figure 2. Despite considerable differences in design from the purely spatial Frame Baseline, to the recurrent LRCN,

the full spatiotemporal Video Swin Transformer, and the geometry-based ReStraV, every model exhibits the same pattern: strong performance on sources visually similar to the training distribution, and systematic failure on sources that are not. This uniformity strongly suggests that the failure is not architectural in origin. The models are not underpowered or poorly trained; they are learning the wrong thing.

What each model appears to learn is a set of source-specific signatures: discriminative features that distinguish the generators and real-world collections present at training time, rather than a domain-general representation of what makes a video AI-generated. The model effectively learns to ask "does this video look like DynamiCrafter, SEINE, or ZeroScope" rather than "does this video exhibit generation artefacts".

When a test source resembles a training source, the model's decision boundary transfers and performance is high. When it does not, the boundary is irrelevant and performance collapses. This trend commonly referred to as shortcut learning [Geirhos et al. 2020] is particularly severe here because of the single class source structure of IVY-Fake: every source is entirely real or entirely fake, meaning a model can achieve high training accuracy by learning source identity rather than the real/fake distinction.

5.2 The Efficacy of Ensemble Methods

The success of the Random Forest Ensemble provides the most promising architectural response to source signature learning. Because individual models overfit to different subsets of artefacts (e.g., ReStraV focusing on geometric trajectories while LRCN focuses on sequential temporal inconsistencies), their blind spots are largely non-overlapping.

The Ensemble leverages these orthogonal strengths, culminating in a superior F1 score and a marked improvement in recall. This suggests that while a single unified concept of a synthetic video may be currently unreachable for a standalone CNN, a multifaceted approach can successfully approximate it.

5.3 The Role of Pretrained Representations

The results offer a nuanced view of the value of pretrained weights for this task. The two highest-performing models (the LRCN and the Video Swin Transformer) both rely on pretrained backbones. Conversely, the newly introduced 3DCNN and standard (2+1)D CNN lack this advantage, and exhibited the poorest generalisation capabilities (0.6433 and 0.6905 test accuracy, respectively).

This might suggest that pretrained features still hold a generalisation advantage, however all pretrained models still exhibit the same source-signature failure pattern. Pretraining on ImageNet or Kinetics provides richer initial feature representations, but these features are still fine tuned on a training set with limited source diversity, and the resulting specialisation to training sources occurs regardless. The pretrained models do not learn a more general concept of AI-generated content; they simply start from a better-initialised feature space before overfitting to the available source signatures.

5.4 Comparison with the State of the Art

The IVY-Fake paper introduces Ivy-xDetector but reports no CNN baseline results, having moved directly to a multimodal large language model approach. This study provides an empirical comparison of CNN-based architectures against the IVY-Fake benchmark under the same evaluation protocol, establishing a concrete reference point against which future methods can be measured.

Across all evaluated models, the best test accuracy achieved is 77.80% (LRCN), compared to the 86.4% reported by Ivy-xDetector. Ivy-xDetector fine-tunes a pretrained Qwen2.5-VL-3B multimodal large language model (MLLM) using supervised fine-tuning on Gemini-generated chain-of-thought reasoning annotations, followed by group relative policy optimisation (GRPO). The 8.6 percentage point gap between the strongest CNN result and Ivy-xDetector reflects a fundamental difference in the nature of the visual priors available to each approach, rather than a failure of the CNN architectures themselves. A model pretrained on internet-scale multimodal data has been exposed to an enormously diverse range of visual content including many of the real-world video styles and generation characteristics that appear in the IVY-Fake test set. Its representations generalise better not because of any algorithmic superiority, but because its effective training distribution is vastly broader than the IVY-Fake training split alone.

This distinction is significant. The source-signature learning failure documented throughout this study is a consequence of limited source diversity at training time. Ivy-xDetector sidesteps this problem not by solving it architecturally, but by beginning with a backbone already exposed to the breadth of content that CNN models trained only on IVY-Fake must learn from scratch. The results collectively suggest that for open set AI generated video detection, the quality and diversity of pretrained visual priors is a more decisive factor than architectural design. Closing the gap to MLLM based methods from a CNN starting point would likely require either substantially more diverse training data, or supervision signals that encourage reasoning about generation artefacts rather than source specific pattern matching.

5.5 Practical Implications

The results have important implications for real-world AI-generated video detection. In practice, detectors are likely to encounter new text-to-video and image-to-video generators that were not available during training. Therefore, high validation accuracy alone is not sufficient evidence of deployment reliability. Aggregate metrics can also hide severe source-level failures. Although the best models achieve reasonable overall test performance, Table 4 shows that performance varies greatly between sources. For example, LRCN achieves 0.9158 accuracy on Crafter and 0.9310 on Gen2, but drops to 0.7600 on HotShot and 0.1543 on self_craw. More broadly, all models remain below 0.25 accuracy on self_craw. This uneven performance would be problematic in high-stakes settings such as content moderation, journalism, digital forensics, or platform safety.

The precision-recall imbalance further affects deployment. As shown in Table 3, the strongest models achieve high precision, such as 0.9774 for LRCN and 0.9523 for the Ensemble, but much lower recall,

at 0.5675 and 0.6214 respectively. This means that when a model predicts a video as fake, the prediction is usually reliable, but many fake videos are still incorrectly classified as real. Therefore, these models are more suitable for flagging suspicious videos for further human review than for making fully automated decisions.

6 CONCLUSION

This study establishes a rigorous empirical baseline for automated AI-generated video detection on the IVY-Fake dataset. By evaluating a comprehensive array of architectures under a strict open-set protocol, we demonstrated that standalone CNN-based methodologies currently plateau below the 80% accuracy threshold. While the LRCN proved the most effective single architecture (77.80% accuracy), the Ensemble Random Forest model yielded the most robust discriminative performance, achieving the highest MacroF1 score (0.7252) and highest recall (0.6214) by successfully aggregating orthogonal features from diverse networks

The central finding of this work is that source-signature learning is the dominant failure mode for CNN-based approaches on this task, and that it is independent of architectural choice. Every evaluated model regardless of whether it uses spatial aggregation, recurrent temporal modelling, factorised spatiotemporal convolution, transformer-based attention, or geometry-based trajectory features learns to recognise the specific generators and real-world sources present at training time rather than a generalised representation of AI-generated content. This manifests as reliable detection of generators that resemble training sources, and systematic failure on generators that do not. Per-source analysis confirms that the visual and temporal similarity of a test source to the training distribution is a stronger predictor of model performance than any architectural property.

Comparison with Ivy-xDetector, the MLLM-based method proposed alongside the dataset, reveals an 8.6 percentage point gap (77.80% vs 86.40%). This gap is attributable to the breadth of visual priors available to a model pretrained at internet scale, rather than to any algorithmic advantage. The CNN baselines established here provide a reference point that was absent from the original paper, and demonstrate that conventional trained-from-scratch or fine-tuned approaches plateau below 80% under open-set conditions on this benchmark.

6.1 Limitations

The primary limitation of this work is tied to the constrained source diversity of the IVY-Fake training split, which is an inherent property of the benchmark rather than a methodological choice. Training on a fixed set of generators and real-world sources makes it structurally difficult to learn source-agnostic features, regardless of architecture. However, the superior recall of the Ensemble framework actively demonstrates that disparate architectures capture genuinely different aspects of the real versus synthetic dichotomy, proving that multi model consensus is a viable pathway to improved robustness. All models were trained and tested on the IVY-Fake dataset only. There was no cross dataset generalisation on other datasets such as FaceForensics++ [Rössler et al. 2019] or Deepfake Detection Challenge [Dolhansky et al. 2020]. Most models are also trained from scratch whereas fine-tuning on a pre-trained backbone would most likely yield better results, and testing the models on external datasets would provide more certainty on the accuracy of the networks.

The image size of the frames and clips could be a limitation since everything has been downsampled to a resolution of 224×224, so compression artefacts may be lost to the resolution. There is also a metadata leakage risk: if the real and fake videos share systematic differences in encoding, the resulting models may exploit these features instead of learning actual forgery clues.

Another limitation was the Aire HPC infrastructure which was used to run the jobs that trained the models. Due to constraints on the hardware, we weren't able to test all combinations of training parameters and so most of the model parameters were fixed. There was a 48 hour time limit on the length of a job and with the dataset consisting of over 100,000 videos, only 10 epochs were able to run in this time frame, and only one model could be trained at a time due to resource constraints. Training was conducted on a single GPU node consisting of a Dell R7615 server with 3x NVIDIA L40S 48GB GPUs. Despite the available hardware, training was restricted to a singular node. Furthermore, the storage was a Dual 480GB NVMe drive which is quite reasonable, however, with 100,000 videos, the I/O could have been a bottleneck in terms of loading the data in order to train models.

6.2 Future Work

Future research must prioritise decoupling detection mechanisms from source-specific signatures.

Advanced Data Augmentation: Simulating unseen generator artefacts (e.g., through frequency domain perturbations or latent style transfer) to force models to learn broader features.

Optimised Ensembling: Expanding on the success of the Random Forest by exploring dynamic weighting or meta learning frameworks that adaptively rely on specific sub models depending on the input's spatial or temporal characteristics.

Prior Expansion: Leveraging foundational models with highly diverse, pre-existing multimodal priors to better contextualise and identify emergent synthetic anomalies.

Identifying Hard Sources: Per-source analysis consistently identifies certain sources - notably HotShot and `self_craw` - as being resistant to detection across all architectures, a pattern also seen in prior benchmarking on IVY-Fake [Changjing Jiang 2025]. Future work would investigate hard sources to determine whether their difficulty is due to a specific artefact being absent or under-represented in training data, helping strengthen the data collection strategy for improving generalisation.

Training Source Diversity: Results indicates that the primary bottleneck for generalisation is not model complexity, LRCN outperforms the significantly heavier Video Swin transformer on the test set. This suggests that what sources are seen during training matter more than how expressive the model is. An extension would

be to progressively expand the training source set and measure how cross-source generalisation scales with diversity, determining whether a threshold exists beyond which additional source variety yields diminishing returns. This would provide guidance for dataset curation in future detection benchmarks.

ACKNOWLEDGMENTS

Abbie Hewitt would like to thank our supervisor, Muhammad Zeeshan Babar, for keeping us on track when it felt like there was no end in sight, and for supporting us throughout the development of this project.

Alex Northam would also like to thank our supervisor, as well as his friends and family both in Leeds and elsewhere for being there to offer moral support when it mattered most.

Ayomide Soruna would like to thank our supervisor for his patience and technical guidance throughout the project, and his family for their constant encouragement during the long stretches of dataset wrangling and pipeline debugging.

Chien-Wei Tung would like to thank our supervisor for his thoughtful feedback during weekly meetings, and his teammates for the collaborative spirit that made this project both productive and enjoyable.

Hsin Jee Sum would like to thank our supervisor for his continued guidance throughout the project, and his teammates for the dedication and adaptability that made the comparative study possible. He would also like to thank his family and friends for their unwavering support during the most demanding stretches of the work.

REFERENCES

Naphtali Rishe Arpan Mahara. 2025. *Methods and Trends in Detecting AI-Generated Images: A Comprehensive Review.* Ph. D. Dissertation. Florida International University.

Leo Breiman. 1996. Bagging predictors. *Mach. Learn.* 24, 2 (Aug. 1996), 123–140.

Zhonghao Zhang Chenyang Si Fengchang Yu Wei Peng Xinbin Yuan Yifei Bi Ming Zhao Zian Zhou Caifeng Shan Changjing Jiang, Wenhui Dong. 2025. *Ivy-Fake: A Unified Explainable Framework and Benchmark for Image and Video AIGC Detection.* Technical Report.

Brian Dolhansky, Joanna Bitton, Ben Pflaum, Jikuo Lu, Russ Howes, Menglin Wang, and Cristian Canton Ferrer. 2020. The DeepFake Detection Challenge (DFDC) Dataset. (June 2020). arXiv:2006.07397 [cs.CV]

Jeff Donahue, Lisa Hendricks, Sergio Guadarrama, Marcus Rohrbach, Subhashini Venugopalan, Trevor Darrell, and Kate Saenko. 2015. Long-term recurrent convolutional networks for visual recognition and description. 2625–2634. https://doi.org/10.1109/CVPR.2015.7298878

Adam Paszke et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. https://arxiv.org/abs/1912.01703

Moncef Garouani, Ayah Barhrhouj, and Olivier Teste. 2025. XStacking : An effective and inherently explainable framework for stacked ensemble learning. *Information Fusion* 124 (2025), 103358. https://doi.org/10.1016/j.inffus.2025.103358

Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard S. Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. 2020. Shortcut Learning in Deep Neural Networks. *CoRR* abs/2004.07780 (2020). arXiv:2004.07780 https://arxiv.org/abs/2004.07780

Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation* 9 (11 1997), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Christian Internò, Robert Geirhos, Markus Olhofer, Sunny Liu, Barbara Hammer, and David Klindt. 2026. AI-Generated Video Detection via Perceptual Straightening. arXiv:2507.00583 [cs.CV] https://arxiv.org/abs/2507.00583

Yoshua Bengio Hod Lipson Jason Yosinski, Jeff Clune. 2014. How transferable are features in deep neural networks? *Advances in Neural Information Processing System* (2014).

Yann Lecun, Patrick Haffner, Yoesoep Rachmad, and Leon Bottou. 1998. Gradient-Based Learning Applied to Document Recognition. *Proc. IEEE* 86 (12 1998), 2278 – 2324. https://doi.org/10.1109/5.726791

Ze Liu, Jia Ning, Yue Cao, Yixuan Wei, Zheng Zhang, Stephen Lin, and Han Hu. 2021. Video Swin Transformer. *CoRR* abs/2106.13230 (2021). arXiv:2106.13230 https://arxiv.org/abs/2106.13230

Ze Liu, Jia Ning, Yue Cao, Yixuan Wei, Zheng Zhang, Stephen Lin, and Han Hu. 2022. Video Swin Transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).* 3202–3211.

Zeno Geradts Marissa Koopman, Andrea Macarulla Rodriguez. 2018. *Detection of Deepfake Video Manipulation.* Ph. D. Dissertation. University of Amsterdam Netherlands Forensic Institute.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *J. Mach. Learn. Res.* 12, null (Nov. 2011), 2825–2830.

Andreas Rössler, Davide Cozzolino, Luisa Verdoliva, Christian Riess, Justus Thies, and Matthias Nießner. 2019. FaceForensics++: Learning to Detect Manipulated Facial Images. In *IEEE International Conference on Computer Vision (ICCV)*.

Connor Shorten and Taghi Khoshgoftaar. 2019. A survey on Image Data Augmentation for Deep Learning. *Journal of Big Data* (2019).

Du Tran, Lubomir Bourdev, Rob Fergus, Lorenzo Torresani, and Manohar Paluri. 2015. Learning Spatiotemporal Features with 3D Convolutional Networks. arXiv:1412.0767 [cs.CV] https://arxiv.org/abs/1412.0767

Du Tran, Lubomir D. Bourdev, Rob Fergus, Lorenzo Torresani, and Manohar Paluri. 2014. C3D: Generic Features for Video Analysis. *CoRR* abs/1412.0767 (2014). arXiv:1412.0767 http://arxiv.org/abs/1412.0767

Du Tran, Heng Wang, Lorenzo Torresani, Jamie Ray, Yann LeCun, and Manohar Paluri. 2017. A Closer Look at Spatiotemporal Convolutions for Action Recognition. (Nov. 2017). arXiv:1711.11248 [cs.CV]

Du Tran, Heng Wang, Lorenzo Torresani, Jamie Ray, Yann LeCun, and Manohar Paluri. 2018. A Closer Look at Spatiotemporal Convolutions for Action Recognition. arXiv:1711.11248 [cs.CV] https://arxiv.org/abs/1711.11248

Sheng-Yu Wang, Oliver Wang, Richard Zhang, Andrew Owens, and Alexei A. Efros. 2019. CNN-generated images are surprisingly easy to spot... for now. *CoRR* abs/1912.11035 (2019). arXiv:1912.11035 http://arxiv.org/abs/1912.11035

A PROJECT MANAGEMENT

A.1 Supervisor and Theme

The theme of this project emerged from the group's shared interest in artificial intelligence, particularly recent developments in generative AI and AI-generated media. At the beginning of the project, we considered several possible directions within AI, including topics related to large language models, however AI generated video is a timely, emerging technology that has become increasingly visible due to rapid progress in text to video generation systems. For this reason we decided to select it as the main theme of our project.

After being assigned to our supervisor, Dr Muhammad Babar, whose expertise is in computer vision and deep learning, the project direction became more focused. During the early discussions, the group considered several related topics, including deepfake detection, detection of videos generated from text-to-video models, and detection of videos generated from image-to-video models. However, with Dr Babar's guidance, we decided to focus more broadly on AI-generated video detection across different generation sources, as this was considered a more challenging and substantial direction that better matched the scope of an eight-month group project.

This direction provided a strong match between the group's interest in modern AI systems and the supervisor's expertise in vision based methods. It also gave the project a clear technical challenge: investigating how different deep learning architectures can distinguish real videos from AI-generated ones. The final topic was both practically relevant and suitably complex for a group based research project, as AI-generated videos are becoming increasingly realistic and accessible. Video detection is technically challenging because models are required to capture both spatial artefacts within individual frames and temporal inconsistencies across sequences of frames in order to perform optimally. This made the topic suitable for comparing multiple approaches, such as CNN-based models, recurrent models, transformer-based models, and trajectory-based methods.

Regular contact was maintained with Dr Babar throughout the project. During the early research and planning phase, meetings were generally held every two weeks, before becoming weekly during the main development and experimentation phase. The meetings usually followed a progress-review structure: the group summarised recent work, raised technical issues or blockers, received feedback from Dr Babar, and agreed on actions for the next stage. Their main goal was to refine the project scope, review dataset and methodology choices, address computational requirements, and ensure the project remained feasible and on track.

Dr Babar's guidance helped shape the project's experimental direction. In particular, he emphasised the importance of documenting training details, changing experimental parameters systematically, reporting numerical performance changes clearly, and using appropriate visualisations to support discussion. This helped the group keep the project focused, feasible, and academically grounded.

A.2 Group Organisation

Our group consisted of five members and adopted an Agile-inspired project management approach due to the exploratory nature of the project and the need to develop multiple AI-generated video detection models in parallel. Since model experimentation involved uncertainty in dataset selection, computational constraints, and varying implementation difficulty, the project was organised into short development sprints with regular review meetings. This was considered more appropriate than a rigid linear plan, as it allowed priorities to be adjusted throughout the project.

Responsibilities evolved throughout the project. Initially, the team worked collectively on literature review, project planning, and identifying a suitable dataset. After the project scope had been finalised, members moved into more specialised roles by developing separate model architectures and conducting independent experiments. In the final stage, efforts shifted towards combining results, writing the report, and ensuring consistency across all sections.

The team established a shared GitHub repository for version control, documentation, and collaborative development. GitHub Issues and a Kanban board were used to manage tasks, assign responsibilities, and track progress throughout the project. Tasks were organised into categories such as data preprocessing, model implementation, experimentation, report writing, and presentation preparation. In addition, the GitHub Wiki was used to share documentation including meeting notes, planning materials, progress updates, and technical guidance.

The group held regular internal meetings to coordinate progress and discuss blockers. Formal supervision meetings were also held regularly with our supervisor. During the early research and planning phase, meetings were typically held on a bi-weekly basis, before increasing to weekly meetings during the main development phase and becoming more frequent near major deadlines. These meetings were used to review progress, receive feedback, refine methodology,

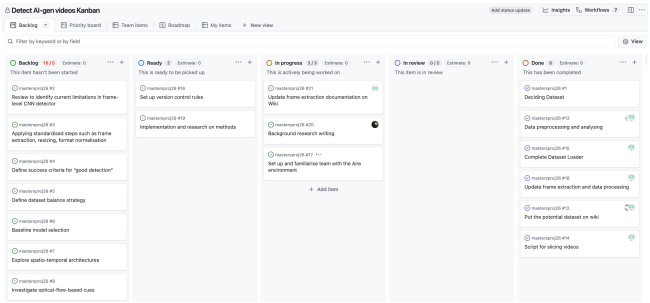


Fig. 3. GitHub Projects Kanban board used in February to track task progress and coordinate work across the team.

and agree on next actions. Formal meeting minutes were recorded after meetings to improve accountability and continuity. These notes summarised agendas, progress updates, risks, and agreed actions. This proved particularly useful when coordinating multiple independent workstreams and ensuring that absent members could remain informed.

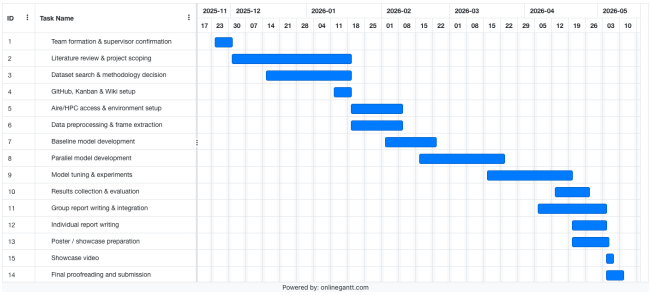
As the project progressed, the team adapted priorities when required. Additional emphasis was placed on experimentation quality, numerical reporting of results, and report integration as deadlines approached. During the final phase, greater focus was placed on report writing, visualisation, and integrating all outputs into a coherent final submission. Overall, the use of version control, structured task management, regular meetings, and documented actions helped the team remain organised while managing a technically demanding project with multiple parallel components.

A.3 Planning and execution

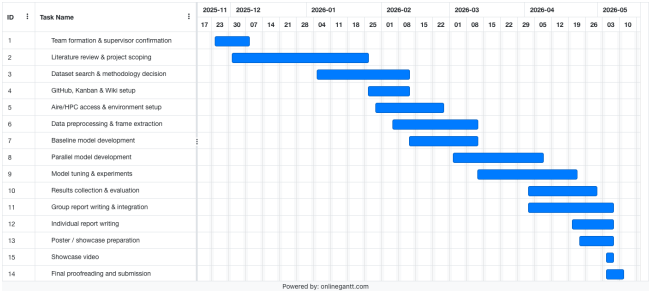
The following Gantt charts compare the initial planned project timeline and the actual execution timeline. The initial project plan was adjusted to use the same task categories as the actual execution chart, allowing a clearer comparison between what was originally planned and what occurred during the project. Figure 4(a) shows the initial plan, while Figure 4(b) shows the actual project execution.

The initial plan assumed that the project would progress in a relatively linear sequence, beginning with team formation, literature review, project scoping, dataset selection, infrastructure setup, preprocessing, model development, experimentation, evaluation, and final deliverable preparation. This plan was reasonable at the start of the project because the main project stages were clear, and it provided a useful structure for assigning responsibilities and setting early milestones.

In practice, the actual timeline followed the same broad structure but involved more overlap between tasks. This was mainly due to the exploratory nature of AI-generated video detection and the need to refine the dataset, methodology, and computing environment before full experimentation could begin. During the early stage, literature review and project scoping took longer than initially planned because the group considered several possible directions before narrowing the focus to AI-generated video detection across different



(a) Initial Gantt chart based on the project planning document.



(b) Gantt chart reflecting the actual project execution.

Fig. 4. Planned vs. actual project timelines as Gantt charts.

generation sources. Additional time was also needed to identify a suitable dataset and arrange access to the Aire high-performance computing environment. These were important dependencies, as large-scale video preprocessing and model training could not be carried out effectively until the dataset and computing environment were ready. As a result, dataset planning, GitHub/Kanban/Wiki setup, Aire setup, and preprocessing overlapped more in the actual timeline than in the original plan.

Once the dataset and computing environment were established, the project moved into parallel model development. Rather than following a single implementation path, different members developed separate architectures, including CNN-based, recurrent, transformer-based, and trajectory-based models. This caused model development, tuning, experimentation, and evaluation to overlap, but it also allowed the team to continue making progress even when individual models faced technical issues, training delays, or validation problems.

Compared with the initial plan, report writing and integration also began earlier and overlapped with the later experimentation phase. This was useful because methodology, training details, results, and evaluation findings could be documented as they were produced, rather than reconstructed at the end of the project. Supervisor feedback also influenced this adjustment, as the team was encouraged to document parameters, report numerical changes clearly, and support the discussion with appropriate tables and visualisations. In the final stage, the team focused on combining results, preparing visualisations, refining discussion sections, completing individual

reports, preparing the poster and showcase material, and proofreading the final submission. The actual timeline therefore became more integrated than the original plan, with experimentation, evaluation, and report writing occurring in parallel rather than as completely separate stages.

Overall, the differences between the planned and actual timelines reflected the technical uncertainty of the project. Some deviations were also caused by time constraints, as the project overlapped with other coursework and assessment deadlines, while the research-oriented nature of the work made it difficult to predict exactly how long dataset selection, model training, and experimentation would take. The original plan provided a useful starting structure, while the actual execution required more flexible scheduling and parallel work. By adapting the timeline, using regular review meetings, and reallocating effort towards experimentation quality and report integration, the group was able to complete the main deliverables while maintaining focus on comparative analysis and experimental reliability.

B SELF-REFLECTION AND SOFT SKILLS ACQUIRED.

B.1 Group Reflection

Throughout this project, the group developed not only technical knowledge of AI-generated video detection, but also important soft skills in communication, planning, teamwork, and adaptability. Since the project involved multiple model architectures being developed in parallel, clear coordination was essential. Each member needed to communicate progress, share results, raise technical problems early, and ensure that their work could later be integrated into the final report and evaluation.

One of the main lessons learned was the importance of structured communication and documentation. Regular meetings, meeting notes, GitHub Issues, the Kanban board, and the Wiki helped the group maintain a shared understanding of progress and responsibilities. This was particularly important because different members were working on separate models, including frame-based, recurrent, spatiotemporal, transformer-based, trajectory-based, and ensemble approaches. Without structured communication and documentation, it would have been harder to keep track of progress across the different model workstreams. Clear records helped the group identify completed tasks, unresolved issues, and results that still needed further checking.

The project also strengthened the group's time-management skills. The work had to be balanced alongside other coursework, individual reports, poster preparation, the showcase video, and personal commitments. As a result, the group had to set realistic short-term goals, adapt deadlines when technical problems occurred, and prioritise tasks that were most important for the final deliverables. This became especially important during the final phase, when experimentation, evaluation, report writing, figure generation, and proofreading overlapped. Tracking tasks through the Kanban board helped the group identify issues more clearly and reassign work during meetings when necessary.

Flexibility was another important lesson learned from the project. In practice, the technical setup did not always progress as expected. For example, gaining access to and setting up the Aire high-performance computing environment took longer than initially planned, which delayed some of the heavier model experiments. Once training began, some models also required substantial running time. Heavier architectures such as the Video Swin Transformer and LRCN could take 24 to 48 hours to train, which reduced the time available for further exploration. There was also a period when the Aire system had technical issues and jobs could not be run, meaning the group had to wait before continuing experiments.

To manage these problems, the group adjusted the project timeline and worked more flexibly. For heavier models such as the Video Swin Transformer and LRCN, smaller numbers of epochs were first used to obtain quicker preliminary results. This allowed the implementation to be checked earlier, gave the group an initial comparison of model behaviour, and helped identify whether longer training runs were worthwhile. Once the preliminary results were promising, longer runs were scheduled on Aire. Checkpoints were also used to save intermediate training progress, so that long-running jobs could be resumed rather than restarted if interrupted.

While long-running jobs were being executed on Aire, the members responsible for the LRCN and Video Swin Transformer continued working on related tasks such as analysing previous outputs, organising results, updating the report, preparing visualisations, and refining the evaluation discussion. This made the workflow more efficient because waiting time during training was still used productively. The original plan provided structure, but the team adapted when computational resources, training time, or technical issues affected progress.

Collaborative problem-solving was also central to the project. Members shared knowledge about preprocessing, Aire/HPC workflows, model training, evaluation metrics, and report presentation. For example, because each architecture was implemented separately, the group had to agree on common metrics such as accuracy, precision, recall, F1 score, AUC, and per-source accuracy. This helped make the final comparison more consistent and reliable. When issues arose, the team discussed possible solutions, asked for supervisor feedback, and adjusted the approach where necessary.

The project also improved the group's academic writing and presentation skills. Since different members contributed different technical sections, the final report required consistency in terminology, formatting, figures, tables, and evaluation discussion. The group learned that writing should be unified across the report and that members should be willing to support each other during the integration process. Presenting the work consistently and logically was important, especially when explaining methodology, parameters, results, and design decisions. Documenting these details continuously while experiments were being performed made them easier to explain clearly later.

Overall, from a management point of view, the project showed that successful technical work depends not only on implementation, but also on coordination, documentation, realistic scheduling, and

risk management. The initial plan provided a useful structure, but the group had to adapt as technical challenges appeared, including dataset preparation, Aire computing constraints, long model training times, and final deliverable integration. Regular meetings, Kanban tracking, GitHub documentation, and supervisor feedback helped the group manage parallel workstreams and keep the project aligned with the final submission. These lessons are directly transferable to future academic and professional environments, where software and AI projects often involve uncertainty, changing requirements, shared infrastructure, and the need to communicate decisions and results clearly.

B.2 Individual Reflections

Hsin Jee Sum (SID: sc22hjs): My main contribution was proposing the project direction, setting up the Aire HPC infrastructure, developing the Video Swin Transformer (Swin3D-B) model, and leading the comparative analysis and evaluation visualisations across all team models. Through this work, I developed stronger skills in independent research and problem-solving, particularly when adapting to cluster-based workflows and overcoming VRAM limitations using Automatic Mixed Precision. I learned the importance of clear resource coordination and sharing intermediate results early, especially in a large group project where individual model outputs must later be integrated into a shared ensemble. I also contributed to the main report and poster, gaining experience in communicating technical methods and results clearly under deadline pressure. This experience will be useful in future technical roles because managing end-to-end pipelines — from infrastructure setup through model development to evaluation — reflects the full-stack responsibilities common in applied ML and data engineering teams.

Ayomide Soruna (SID: sc22as2): My main contribution was designing and implementing the full data preparation and pre-processing pipeline: manifest construction, stratified and leakage-safe dataset splitting, validation and cleaning of corrupted samples, and the runtime pipeline for video decoding, temporal sampling, and tensor construction. Because this pipeline was a shared dependency for every model in the project, correctness and flexibility were critical any failure here would propagate to all downstream results. Through this work I developed practical skills in data engineering at scale, including on-demand processing, data loading, and handling of corrupted inputs on the Aire HPC cluster. I also learned to treat infrastructure as a primary concern: decisions such as how frames are sampled or how splits are constructed directly affect what conclusions can be drawn. This experience is directly applicable to professional ML and data engineering roles, where reliable, reproducible data infrastructure underpins the validity of any model comparison.

Chien-Wei Tung (SID:fy19cwt): My main contribution was developing and evaluating several variants of the ReStraV-based AI-generated video detection pipeline. I also supported project management by taking responsibility for all meeting notes and contributing to the Kanban board and project Wiki. In addition, I contributed to report writing, particularly by writing most of Appendix A and Appendix B.

This project improved my practical understanding of machine learning experimentation, HPC workflows, and model evaluation. I developed stronger skills in reproducibility, hyperparameter tuning, and interpreting metrics beyond simple accuracy. Working within a team under an intensive schedule also improved my communication, collaboration, and project management skills.

We adopted an Agile development approach. At times, some tasks did not progress according to schedule, and specifications were not always clearly documented, which made communication more difficult. This experience helped me understand the importance of clear documentation, shared expectations, and resolving issues early before they affect later stages of the project.

It was also interesting to observe how rapidly AI is developing, with many new approaches emerging in both video generation and fake video detection. Through this project, I explored part of this fast-moving research area and gained a better understanding of current challenges and opportunities. Overall, I greatly enjoyed working on this project with my teammates. It allowed me to develop both technical skills and practical research experience in the current area of AI-generated video detection.

Alex Northam (SID: sc222an): My main contribution was designing the Long-term Recurrent Convolutional Network (LRCN), as well as taking the initial lead on the report writing and producing our presentation video using Manim. Through this work, I developed stronger skills in machine learning experimentation and optimisation, particularly when managing jobs on the Aire HPC cluster and utilising techniques like mixed precision training, partial weight freezing, and dynamic class imbalance handling.

As a frame based model equipped with a recurrent temporal aggregation stage, the LRCN acts as a vital bridge between simpler frame-based approaches and more powerful clip-based models. Through establishing this comparison point, I learned the strategic value of ablation studies in machine learning research; it taught me that deliberately isolating specific architectural components, in this case, the LSTM's temporal modelling is essential for accurately attributing performance gains and understanding exactly why a model succeeds, rather than simply taking the final accuracy metrics at face value. This importance also taught me to prioritise clean, robust design, as the validity of the results would depend on a fair comparison.

Writing the initial background and methodology sections, alongside reviewing and improving the overall report later on in the project and creating the Manim animations, gave me valuable experience in synthesising complex literature and translating technical architectures into accessible narratives. This experience will be highly useful in future technical roles, as the ability to rigorously develop ML pipelines while effectively communicating complex concepts reflects the core responsibilities of professional AI research and engineering.

Abbie Hewitt (SID: sc22ah3): My main contribution was developing and evaluating the 3DCNN and (2+1)D CNN models, as well as developing the final ensemble model. Throughout this project, I gained technical knowledge in setting up convolutional networks

and convolutional blocks, understanding 3D and spatio-temporal convolution architectures, ensemble methods and combining model outputs, deep learning frameworks like PyTorch, working with video and volumetric data. I also gained conceptual understanding of the trade-offs between different CNN architectures, debugging and training deep learning models, and writing up my technical findings clearly. Through the utilisation of the Aire HPC, I became familiar with setting up and running jobs on GPU nodes on high performance architecture and using ssh to access all the saved models, as well as setting up arguments for jobs. Writing the report, reviewing it, and collaborating alongside others gave my valuable skills in communicating the implementation and methodology of a technical project which will be useful in my future career endeavours. Reviewing the ethical considerations of AI-generated content detection helped me to view the wider picture and consider real-world implications of this research project.

C HOW ETHICAL ISSUES ARE ADDRESSED

This project involves ethical considerations because AI-generated video detection is directly related to digital authenticity, misinformation, privacy, and security. As AI-generated videos become more realistic, detection systems may help reduce the spread of misleading or manipulated content. However, such systems must also be used carefully, since incorrect predictions could wrongly classify real videos as fake or fail to detect synthetic videos. Obtaining hidden information, patterns, and relationships between data content gives the models knowledge, and with knowledge comes the power to affect others.

C.1 Dataset Ethics

The dataset used was IVY-Fake [Changjing Jiang 2025], which consists of content sourced from a variety of licenses including most which are under the FLUX.1 [dev] Non-Commercial License, and some content accessed via commercial APIs under their respective usage agreements The datasets within IVY-Fake are freely available for research and non-commercial use. All the data generated by generative APIs are used solely for academic research and non-commercial use, and no content is redistributed for commercial deployment or monetisation purposes. Since this project is purely academic research and not intended for commercial deployment, the licensing and consent of all data used has been considered.

There is potential for some bias within the dataset in regards to the actual content present within the real and synthetic videos. No demographic analysis was completed on the dataset, so there could be an unconscious bias present against certain faces, ethnicities, or demographics which might affect the model's capabilities to correctly identify real versus synthetic videos. This is reflected in the model's highly variable per-source performance, since some sources (e.g. self_craw) were consistently misclassified across models, which could indicate a distributional bias in the training data.

C.2 Potential Misuse

There are ethical concerns surrounding the misuse and dual-use implications of this project. Deepfake and AI-generated content detection research has the consequence of advancing deepfake generation. Understanding which artefacts are key markers for synthetic content reveals what future generative models must improve upon to avoid detection. The same features and techniques that are used to detect synthetic content can be used to produce better and harder to detect generative videos. There is a potential for misuse of the projects findings to clear content that is synthetically generated since the results show that the models do not generalise perfectly to unseen video sources, since the results demonstrate performance degradation on unseen generative sources. No synthetic content was generated by us throughout the duration of this project so our research is purely defensive within the field of generative content detection and does not contribute towards dual-use.

The models should not be treated as fully reliable evidence for determining whether a video is real or fake. The system should be used to support human judgement rather than replace it. Therefore, it is important that the project is transparent and includes responsible disclosure. To mitigate the risks of misuse, model performance, limitations, and failure cases have all been reported transparently. All the methodology and implementation has been documented to support informed and responsible use of AI-generated video detection technology.

D DELIVERABLES

All code developed as part of this project can be found on the following GitHub repository: https://github.com/archivehaze/mastersproj26. Trained models are stored on the Aire HPC architecture under the `/scratch/sc22as2/IVY-Fake/runs/` directory. If access to these models is required, please contact one of the authors.

All training and evaluation scripts required to reproduce the reported results are included in the repository."